



E.E.S.T 1 | 5°3° 2026

Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

Actividad 4: Calcular edad mediante cadenas y punteros

C++

Repositorio de GitHub

<https://github.com/Martuuuuuuuu/CADENASPUNTEROS.git>

Participantes

Martina Araujo, Thiago Goya, Sofia Salaberry

Escuela

Escuela Tecnica N°1 de Vicente Lopez

Curso

Ciclo Superior 5°3° turno tarde

Docentes a cargo

Yamil Ganduglia, York Elias Mansilla Muñoz

Año:

2026



E.E.S.T 1 | 5°3° 2026

Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

Índice

Introducción	3
Estructura de Carpetas	3
Capturas del código fuente	3
Capturas de ejecución	4
Cuestionario	6
Respuestas	8



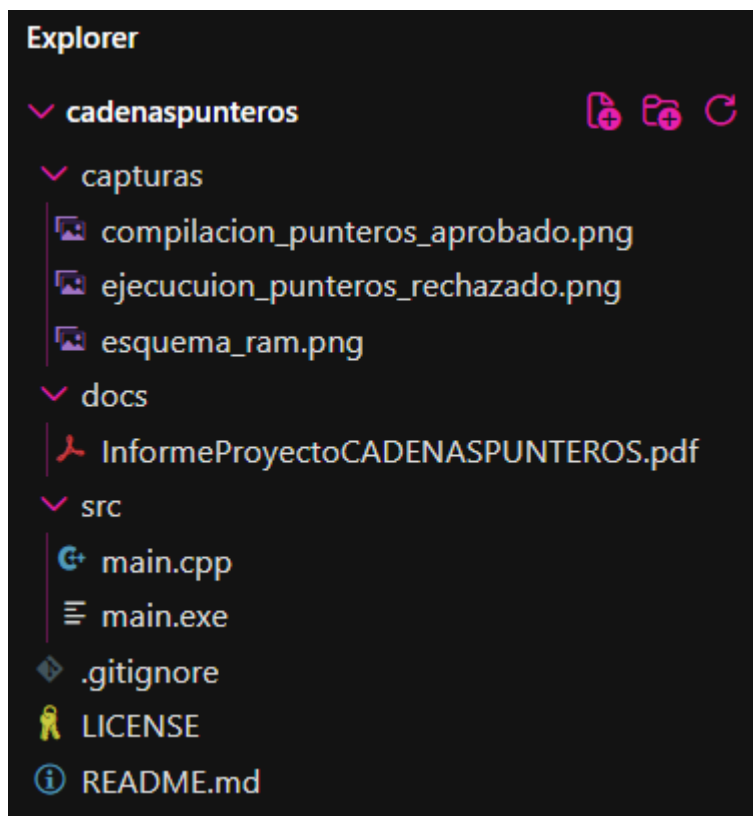
Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

Introducción

En este informe se presenta un programa realizado en C++ que permite controlar el acceso de un usuario según su edad, utilizando punteros con el objetivo principal es comprender cómo un puntero puede almacenar la dirección de memoria de una variable y permitir modificar o consultar su valor directamente.

Estructura de Carpetas





Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

Capturas del código fuente

```
src > main.cpp > main()
1  /*
2  =====
3  Materia: Laboratorio de Programación (LPR)
4  E.E.S.T. N° 1 "Eduardo Ader" – Vicente López
5  Curso: 5° Año 3° División
6  Profesor: Prof. York
7  Archivo: main.cpp
8  Objetivo: control de accesos operando directamente sobre la RAM con punteros.
9  =====
10 */
11
12 #include <iostream>
13
14 using namespace std;
15
16 int main() {
17     int edad = 0;
18     int* p = &edad; // El puntero 'p' guarda la dirección de memoria física de 'edad'
19
20     // =====
21     // REGLA OBLIGATORIA (EVITA EL PLAGIO):
22     // Modifiquen la salida agregando su Nombre y Apellido reales.
23     // =====
24     cout << "===== " << endl;
25     cout << " CONTROL DE ACCESO RAM - ESTUDIANTE: Martina Araujo " << endl;
26     cout << "===== " << endl;
27
28     cout << "=> Ingrese su edad: ";
29
30     // =====
31     // 1. CAPTURA MEDIANTE DESREFERENCIACIÓN (CÓDIGO EXPLICADO)
32     // 🔗 EXPLICACIÓN TÉCNICA:
33     // Usamos 'cin >> *p;' porque el asterisco '*' desreferencia al puntero.
34     // Esto le indica al compilador que debe guardar el número ingresado
35     // directamente en el casillero de memoria original de 'edad'.
```



Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

```
36 // Si pusieramos 'cin >> p;' sin asterisco, intentaríamos sobrescribir
37 // la dirección hexadecimal física de la RAM, provocando un error grave.
38 // =====
39
40 cin >> *p; // Captura el dato guardándolo directo en la memoria apuntada
41
42
43 cout << "\n--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---" << endl;
44
45 // =====
46 // 2. CONDICIONAL CON PUNTERO DESREFERENCIADO
47 // 💡 EXPLICACIÓN TÉCNICA:
48 // Evaluamos el valor alojado en la memoria usando '*p >= 18'.
49 // Si mostramos solo 'p', la consola imprimirá la dirección hexadecimal (ej: 0x61fe1c)
50 // =====
51 if (*p >= 18) {
52     cout << "[ACCESO APROBADO] El usuario es mayor de edad." << endl;
53     cout << "Edad registrada: " << *p << " años." << endl;
54     cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
55 } else {
56     cout << "[ACCESO RECHAZADO] Menor de edad." << endl;
57     cout << "Edad registrada: " << *p << " años." << endl;
58     cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
59 }
60
61 cout << "-----" << endl;
62 return 0; // Finalización exitosa de Windows (Código 0)
63
64 }
```

Capturas de ejecución

Acceso rechazado:

```
PS C:\Users\Martina\Desktop\cadenaspunteros\src> ./main.exe
>>
=====
CONTROL DE ACCESO RAM - ESTUDIANTE: Martina Araujo
=====
=> Ingrese su edad: 5

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO RECHAZADO] Menor de edad.
Edad registrada: 5 años.
Direccion fisica en RAM Hexadecimal: 0xa16a5ff8a4
=====
❖ PS C:\Users\Martina\Desktop\cadenaspunteros\src> 
```



Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

Acceso aprobado:

```
PS C:\Users\Martina\Desktop\cadenaspunteros\src> ./main.exe
>>
=====
CONTROL DE ACCESO RAM - ESTUDIANTE: Martina Araujo
=====
=> Ingrese su edad: 21

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO APROBADO] El usuario es mayor de edad.
Edad registrada: 21 anos.
Direccion fisica en RAM Hexadecimal: 0x6bb8bff754
=====
PS C:\Users\Martina\Desktop\cadenaspunteros\src> 
```

Cuestionario

CUESTIONARIO TÉCNICO DE CONTROL

Respondan las siguientes 3 preguntas dentro del informe PDF final que entregarán en Moodle.

Pregunta 1: Diferencia entre & y *

- **Explicación:** El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- **Pregunta:** ¿Qué valor imprime en pantalla la instrucción `cout << p;` comparada con `cout << *p;`?

Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Intentar escribir `cin >> p;` sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- **Pregunta:** ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar `cin` para modificar el valor de la variable original?

Pregunta 3: Formato Hexadecimal



Actividad 4: Cadenas y punteros

Martina Araujo, Thiago Goya, Sofia Salaberry

- **Explicación:** Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) anteceditas por 0x (ej: 0x61fe1c).
- **Pregunta:** ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Respuestas

1. Al usar “p”, se muestra la dirección de memoria donde está guardada la variable edad, en cambio, cuando usamos “*p”, mostramos el valor que está guardado en esa dirección, por ejemplo, “p” puede mostrar un valor como 0x61fe1c, mientras que “*p” mostraría la edad que ingresamos.
2. Es obligatorio usar “*p” porque “p” guarda la dirección de memoria de la variable edad, al poner “cin >> *p”, el número que ingresamos se guarda en edad a través del puntero, si ponemos “cin >> p”, estaríamos intentando guardar el dato en el puntero y no en la variable original.
3. Las direcciones de memoria se muestran en hexadecimal porque es una forma más corta y sencilla de representar números grandes, además, el hexadecimal se relaciona fácilmente con la forma en que la computadora trabaja con los bits y la memoria.